16 EXPLICIT PROPERTY MAPPING vs SPREADING A DTO (...dto)

When building the object you're actually going to store from a DTO, there are two ways to do it — and only one of them is safe.

```
// src/task/task.service.ts  create()
const newTask: Task = {
  id: this.nextId++,
  title: dto.title,
  description: dto.description,
  done: dto.done ?? false,
};
```

vs the tempting shortcut:

```
const newTask: Task = {
  id: this.nextId++,
  ...dto,          // DON'T do this
};
```

THE KEY FACT: TypeScript TYPES ARE ERASED AT RUNTIME

`create(dto: CreateTaskDto)` only tells the COMPILER what shape to expect. It does NOT strip extra properties from the actual JavaScript object at runtime. The incoming HTTP JSON is still just a plain object by the time it reaches your service — if a client sends extra fields, they're really there on `dto`, TypeScript's type annotation notwithstanding.

Client sends:

```
{
  "title": "Learn NestJS",
  "description": "DTO",
  "done": false,
  "age": 25,
  "role": "admin",
  "isPremium": true
}

dto (at runtime) actually contains ALL six fields, even though the
CreateTaskDto class only declares title/description/done.
```

WHAT ...dto ACTUALLY DOES

`...dto` means "copy every enumerable property currently on this object." So:

```
const newTask = { id: this.nextId++, ...dto };
```

produces:

```
{
  id: 1,
  title: "Learn NestJS",
  description: "DTO",
  done: false,
  age: 25,           // leaked through
  role: "admin",     // leaked through
  isPremium: true    // leaked through
}
```

`role: "admin"`, `isAdmin: true`, `internalNote: "secret"` — anything a client decides to send rides along silently. This is exactly the kind of bug that lets a client set fields they were never supposed to control (privilege escalation via mass assignment).

WHAT EXPLICIT MAPPING DOES INSTEAD

```
const newTask: Task = {
  id: this.nextId++,
  title: dto.title,
  description: dto.description,
  done: dto.done ?? false,
};
```

This says, field by field: "ONLY these four properties end up on newTask — nothing else, no matter what else was in the request." Extra fields like `age`, `role`, `isPremium` are simply never read, so they never make it into `newTask`.

```
HTTP Request -> DTO (may still carry extra fields at runtime)
                  |
          explicit mapping (you pick exactly what to keep)
                  |
              Task object -> stored / returned
```

WHY EXPLICIT MAPPING IS SAFER — FOUR REASONS

A. Prevents unwanted fields Only what you name on the right-hand side is copied. Nothing else gets in by accident.

B. Prevents accidental data exposure / mass assignment A client can't smuggle in `isAdmin: true` or `internalNote` and have it silently stored, because nothing reads those keys.

```
C. Lets you apply defaults inline
   `done: dto.done ?? false` — "if the client sent `done`, use it;
   if they omitted it, default to false." A blind `...dto` gives you
   no place to express this kind of rule.

D. Lets you transform data on the way in
   e.g. `title: dto.title.trim()`, `description: dto.description?.trim()`
   — you fully control what actually lands in the stored object,
   not just which keys survive.
```

ValidationPipe { whitelist: true } vs EXPLICIT MAPPING — DIFFERENT LAYERS

These solve two DIFFERENT problems and are not substitutes for each other:

```
ValidationPipe({ whitelist: true, forbidNonWhitelisted: true })
  Controls the INCOMING REQUEST, before it even reaches your DTO.
      Client -> ValidationPipe -> DTO
  `whitelist: true` strips undeclared properties off the object
  entirely; `forbidNonWhitelisted: true` instead makes it a 400 error
  if any show up. (Neither option is currently set in this project's
  main.ts — `app.useGlobalPipes(new ValidationPipe())` is the bare
  default, so undeclared fields currently DO reach your service on
  `dto` — see theory/15-class-validator-and-class-transformer.txt.)
```

Explicit mapping in the service Controls the OBJECT YOU BUILD/STORE, after the DTO exists. DTO → explicit mapping → Task

BEST PRACTICE: use both, as two independent layers of protection:

```
Client Request
     |
ValidationPipe (whitelist: true) -- rejects/strips extras early
     |
DTO object
     |
Explicit mapping in the service -- only intended fields kept
     |
Task / stored data
```

Even with `whitelist: true` turned on, explicit mapping is still worth keeping — it's a second, independent boundary, and it's also where defaults (`?? false`) and transforms (`.trim()`) naturally belong, which `whitelist` alone doesn't give you.

★ KEY TAKEAWAY

`...dto` copies everything present on the object AT RUNTIME, regardless of what the DTO's TypeScript type claims. Explicit field-by-field mapping copies only what you deliberately choose. For security (no mass assignment), maintainability (defaults/transforms live in one visible place), and a clear boundary between "what a client sent" and "what gets stored" — explicit mapping is the safer default, even when whitelist validation is also in place.

See also: theory/13-dto.txt, theory/15-class-validator-and-class-transformer.txt, theory/14-interface-vs-dto.txt